Bubble Frog Code Map

Every script, function and signal in the game, drawn as flowcharts. Start with the big picture, then follow the three play scenarios, then look up any single script. This matches the fixed versions of `player_1.gd` and `globals.gd` from earlier.

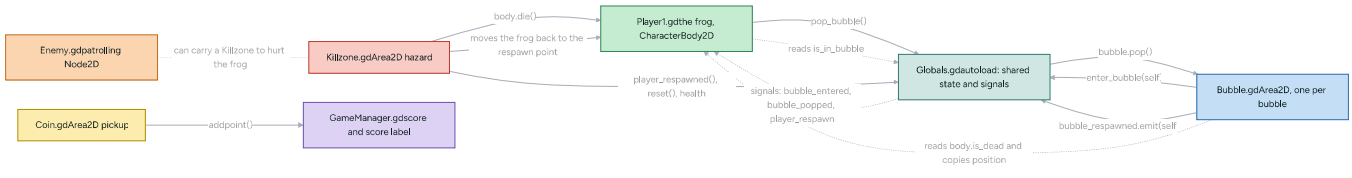
Big picture | Play scenarios | Globals | Player1 | Bubble | Killzone | Coin and GameManager | Enemy | Signal list | Loose ends

`Globals.gd` `Player1.gd` `Bubble.gd` `Killzone.gd` `GameManager.gd` `Coin.gd` `Enemy.gd`

Block colour shows which script owns that line of code. A call that lands in another script takes that script's colour. Dark pill = a function starting. Diamond = a yes or no check. Dashed white pill = a signal being emitted. Solid arrow = a direct call. Dotted arrow = a read or a signal.

Big picture

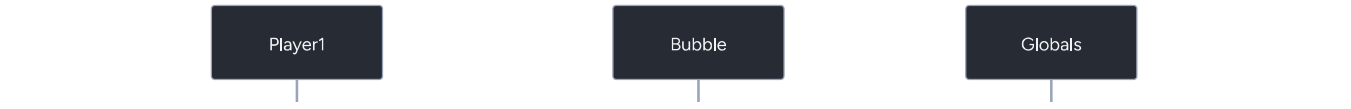
`Globals` is the hub. The frog, the bubbles and the killzone all talk through it, so nothing else needs a direct reference to the frog except the killzone and the bubble.



Three things that happen while you play

Each chart follows one event from start to finish, in order. Dotted arrows are signals.

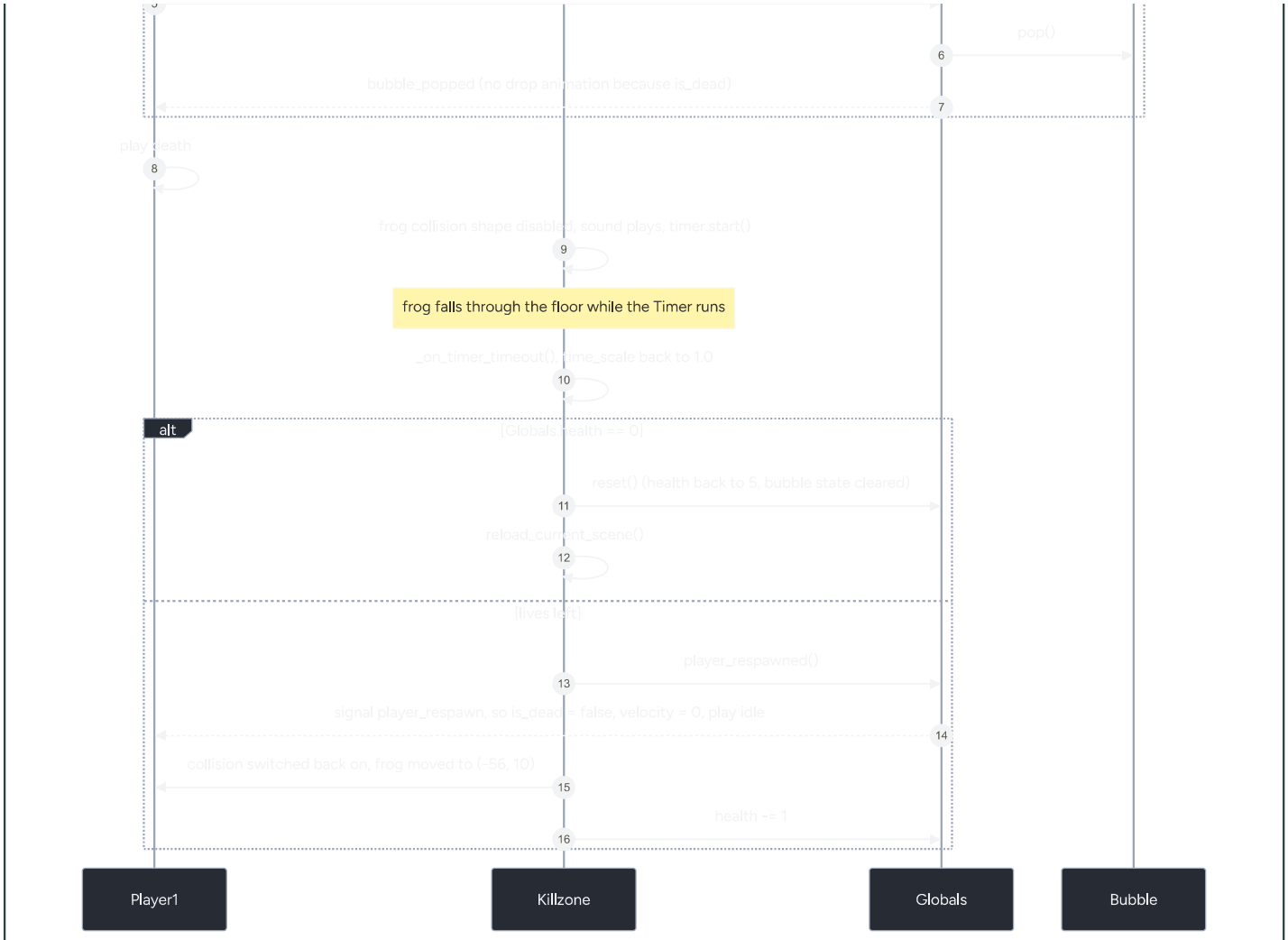
Riding and popping a bubble



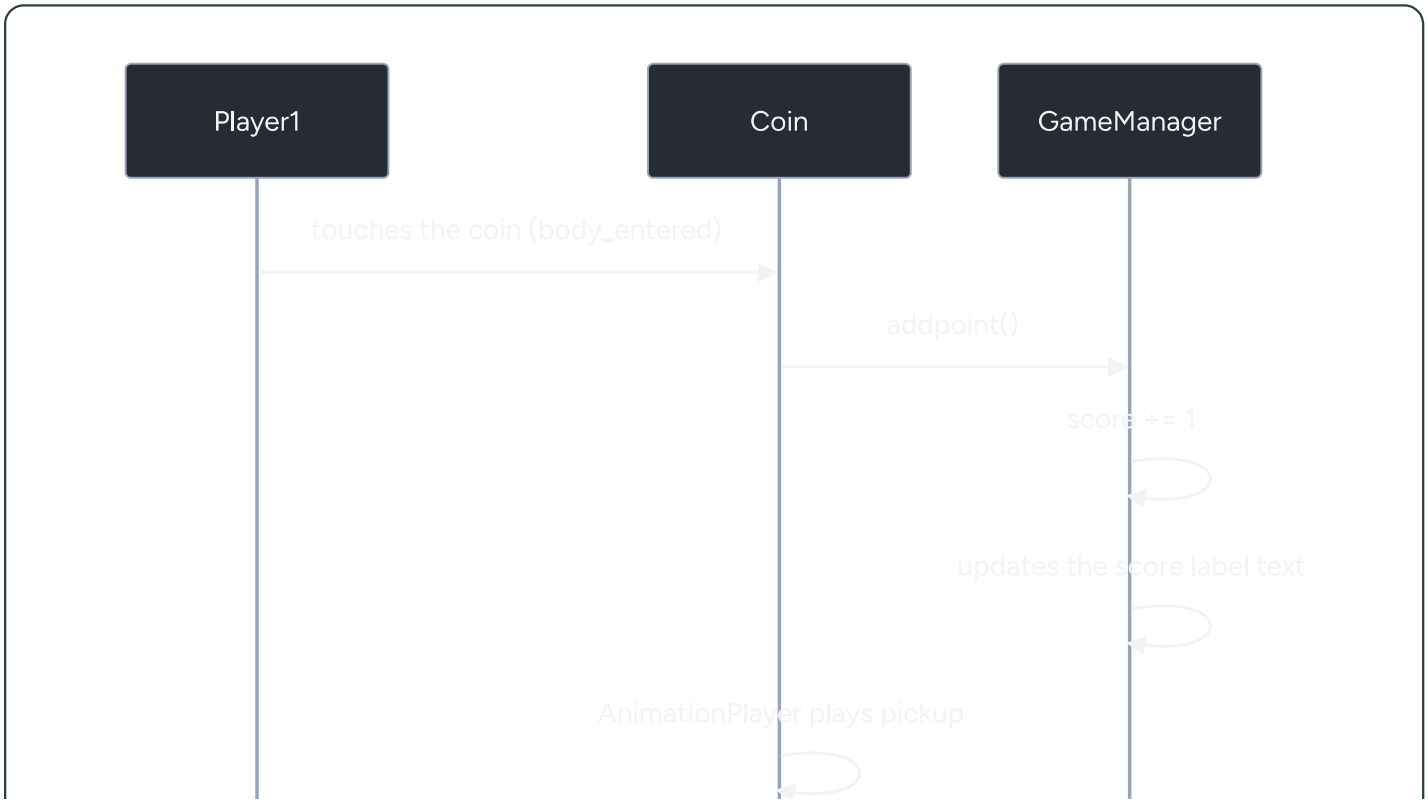


Dying and respawning





Collecting a coin





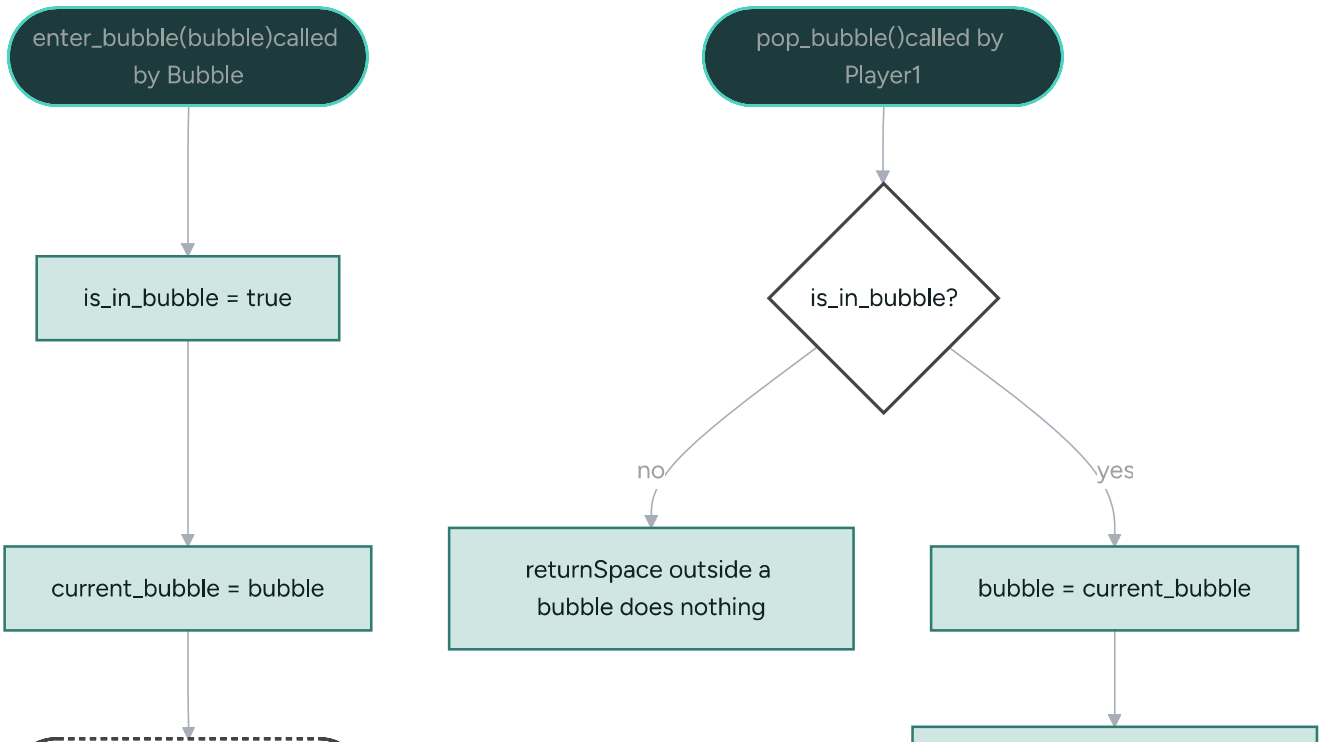
Script by script

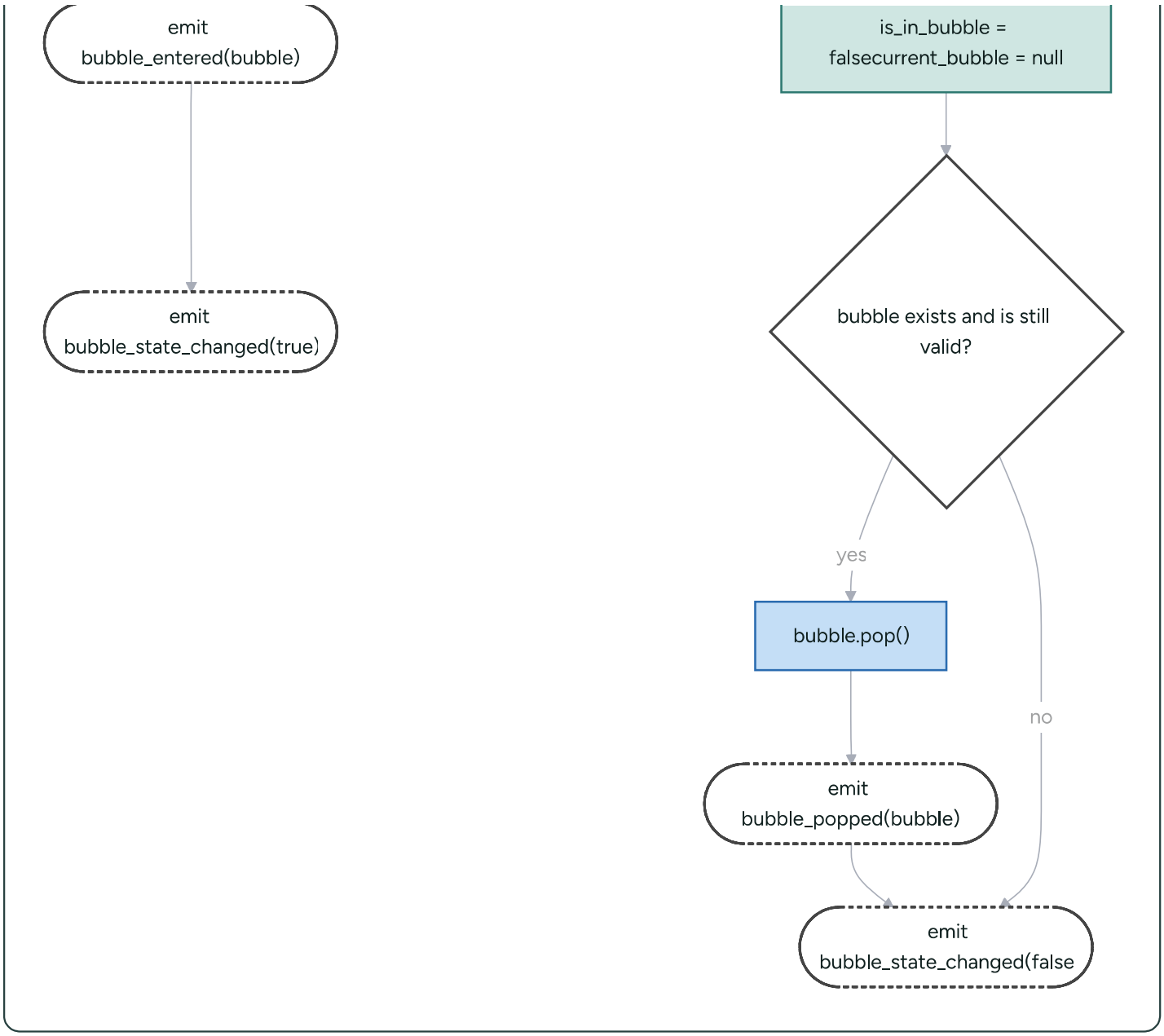
Each script has a short fact sheet and a flowchart for every function. Use these when you want to change or extend one script.

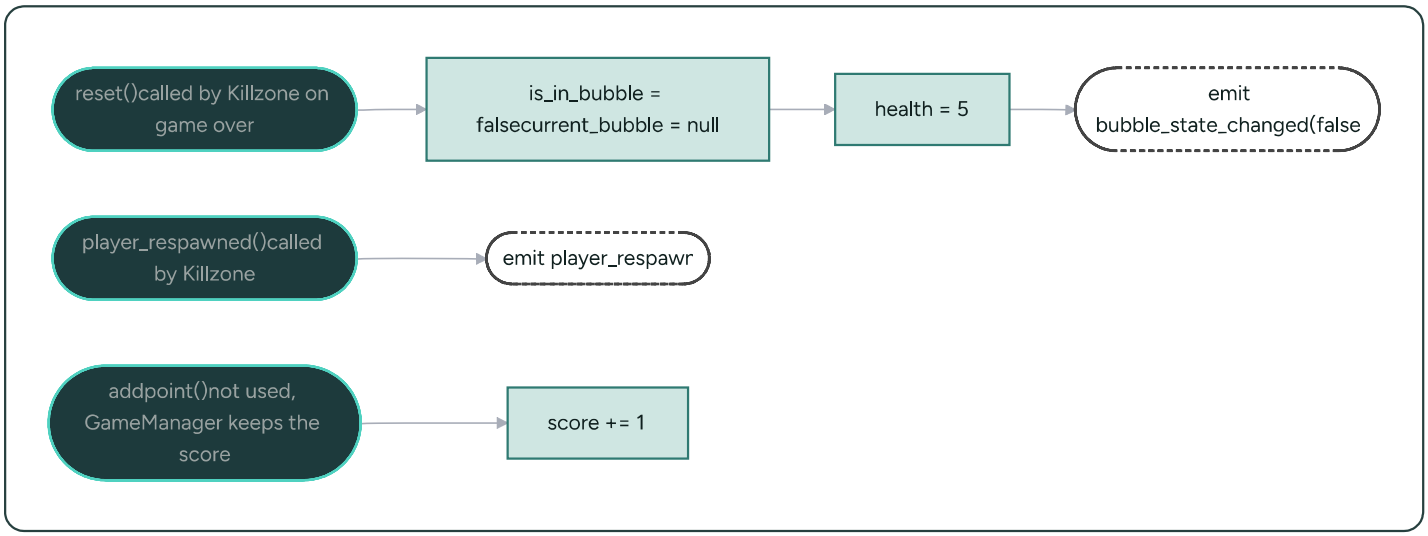
Globals `globals.gd`

An autoload (singleton) that extends `Node`. It holds the bubble state, the lives counter and every shared signal. Any script can call `Globals.something`.

Signals	`bubble_entered(bubble)` `bubble_popped(bubble)` `bubble_respawned(bubble)` `player_respawn` `bubble_state_changed(is_floating)`
Variables	`health = 5` `is_in_bubble = false` `current_bubble = null` `score = 0` (unused) `text` (unused) `respawn` (unused)
Functions	`enter_bubble` `pop_bubble` `reset` `player_respawned` `addpoint`




emit bubble_entered(bubble)
emit bubble_state_changed(true)
is_in_bubble = falsecurrent_bubble = null
bubble exists and is still valid?
yes
no
bubble.pop()
emit bubble_popped(bubble)
emit bubble_state_changed(false

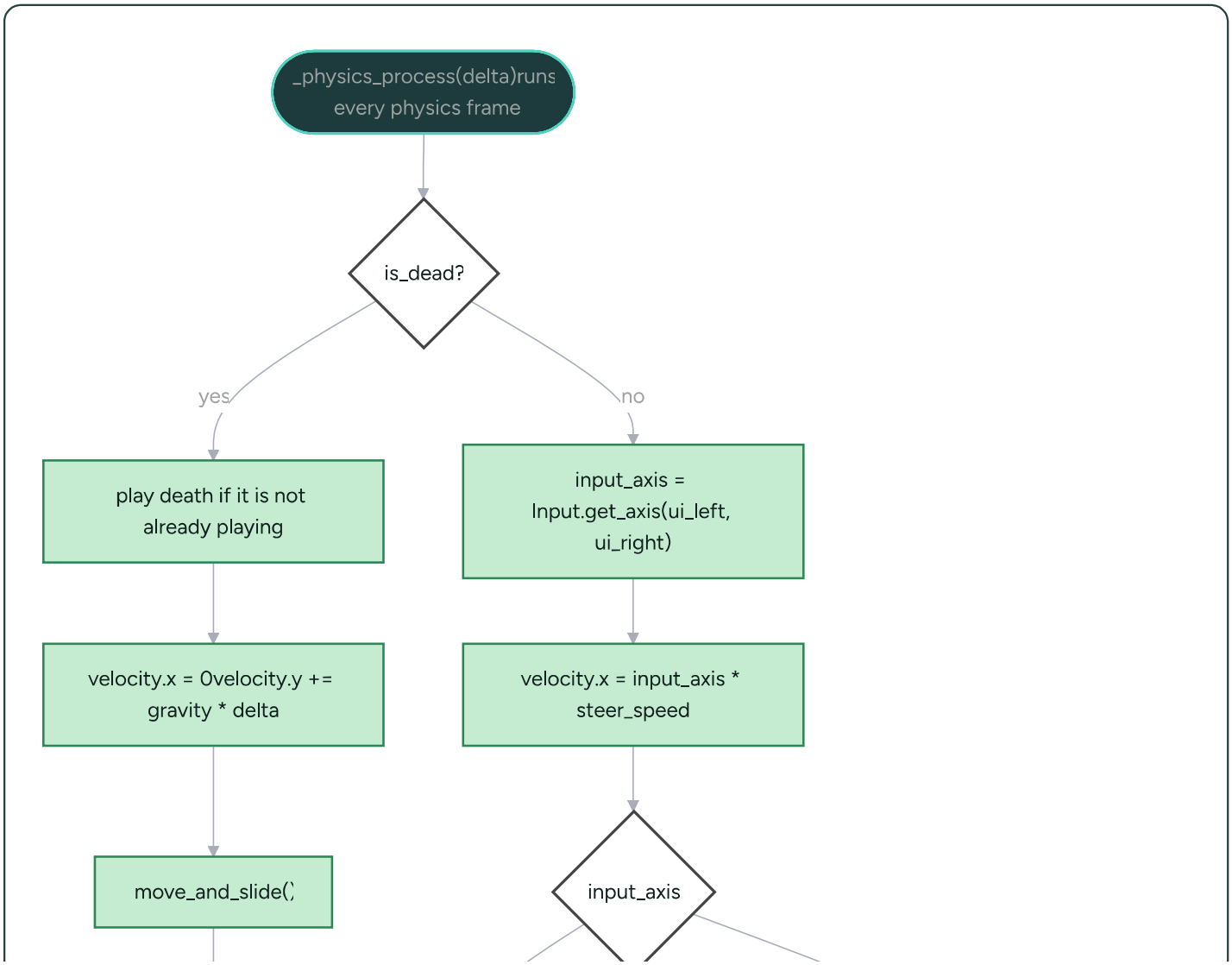

reset()called by Killzone on game over
is_in_bubble = falsecurrent_bubble = null
health = 5
emit bubble_state_changed(false
player_respawned()called by Killzone
emit player_respawr
addpoint()not used, GameManager keeps the score
score += 1

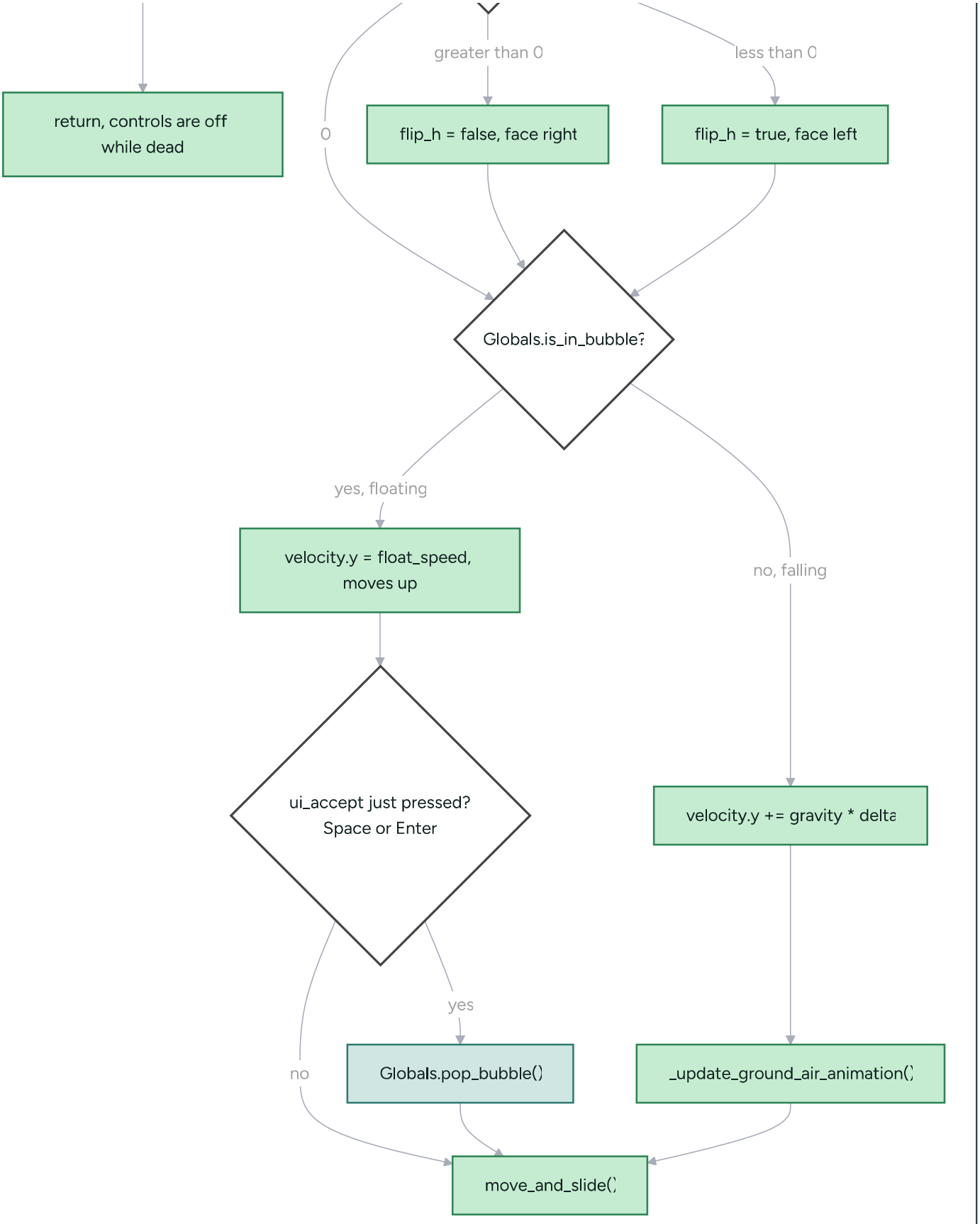
Player1 `player_1.gd`

The frog. Extends `CharacterBody2D`. It steers left and right, floats up inside a bubble, falls with gravity outside one, and plays idle, drop or death animations.

Signals	`entered_bubble(bubble)` `popped_bubble`
Exported	Movement group: `float_speed = -100` `steer_speed = 200` `gravity = 600` `pop_jump_velocity = -200`
Variable	`is_dead` (bubbles ignore the frog while this is true)
Node used	`AnimatedSprite2D` with animations `idle`, `drop`, `death`
Input actions	`ui_left` `ui_right` `ui_accept`
Functions	`_ready` `die` `_physics_process` `_update_ground_air_animation` `_on_bubble_entered` `_on_bubble_popped` `_on_player_respawn` `_on_animated_sprite_2d_animation_changed` `_on_popped_bubble`

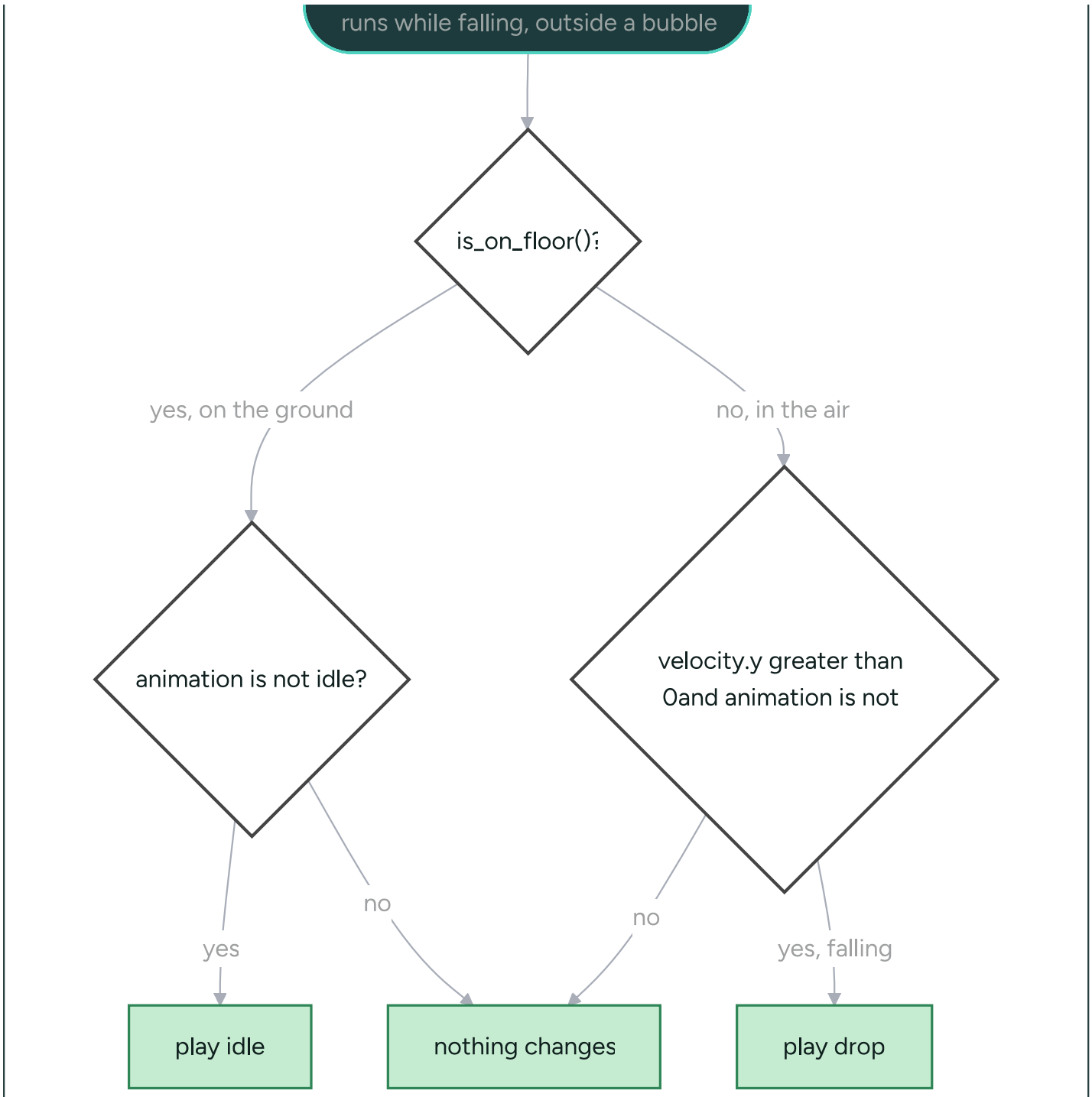
_PHYSICS_PROCESS: THE MAIN LOOP



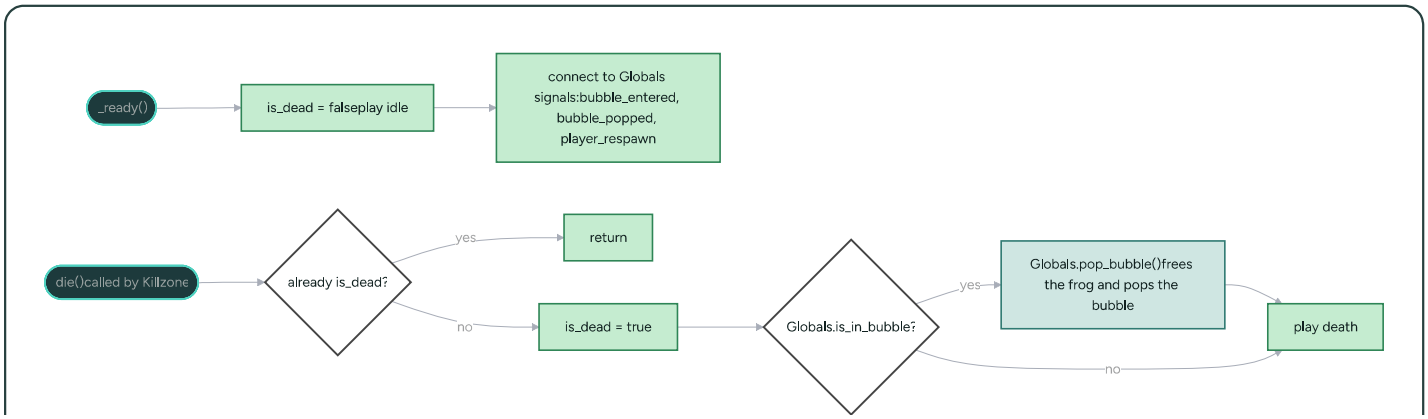


_UPDATE_GROUND_AIR_ANIMATION: IDLE OR DROP

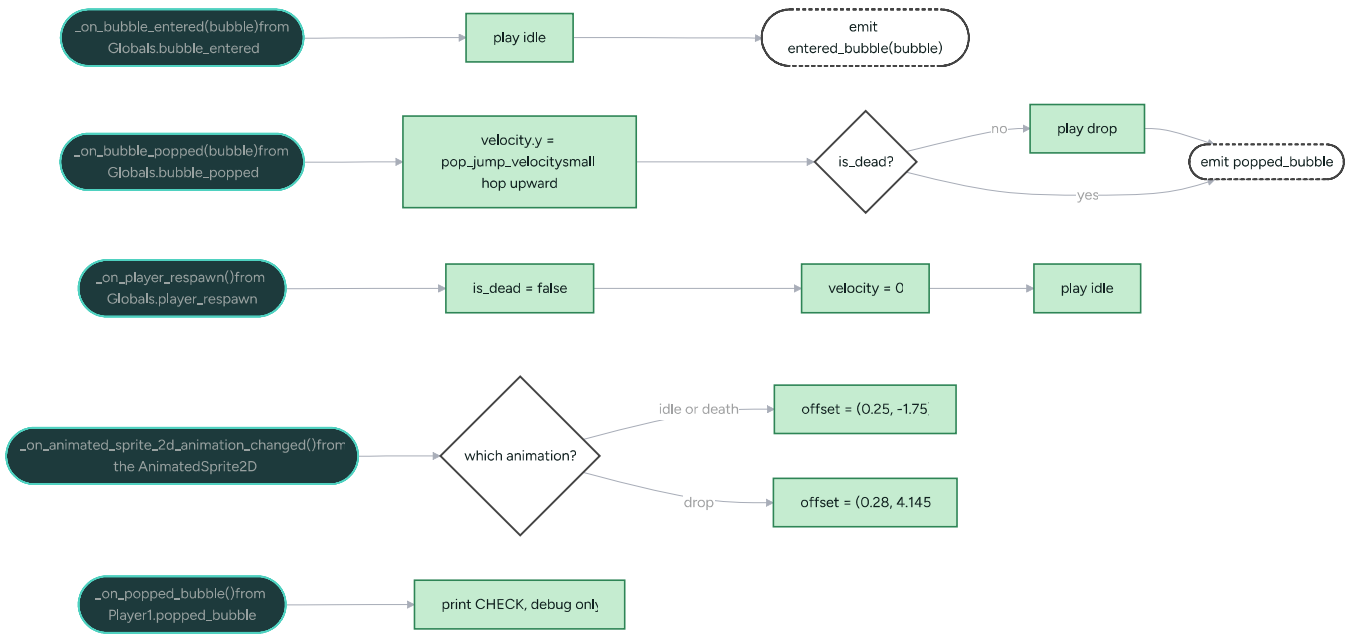
_update_ground_air_animation()only



_READY AND DIE



SIGNAL HANDLERS

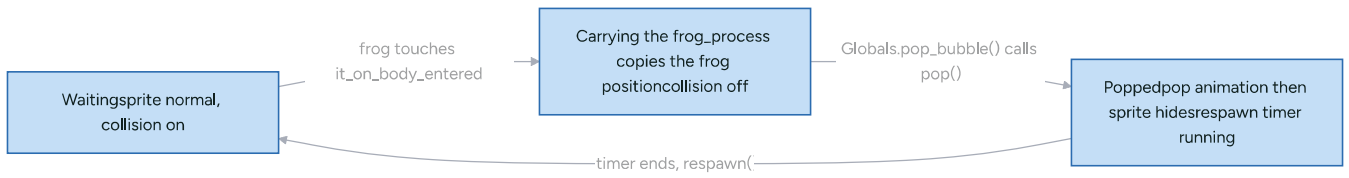


Bubble `bubble.gd`

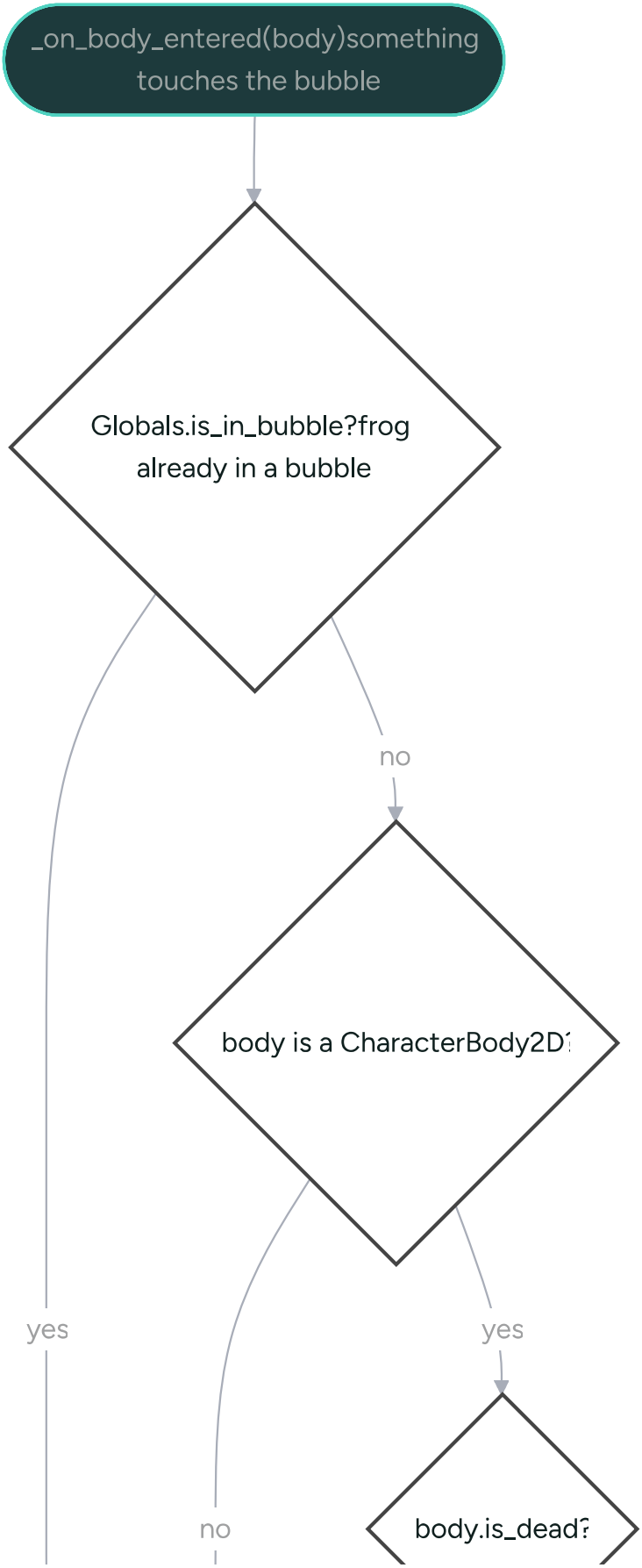
One bubble. `class_name Bubble`, extends `Area2D`. It waits, carries the frog upward, pops, then comes back after a timer. Every bubble in the level runs its own copy of this script.

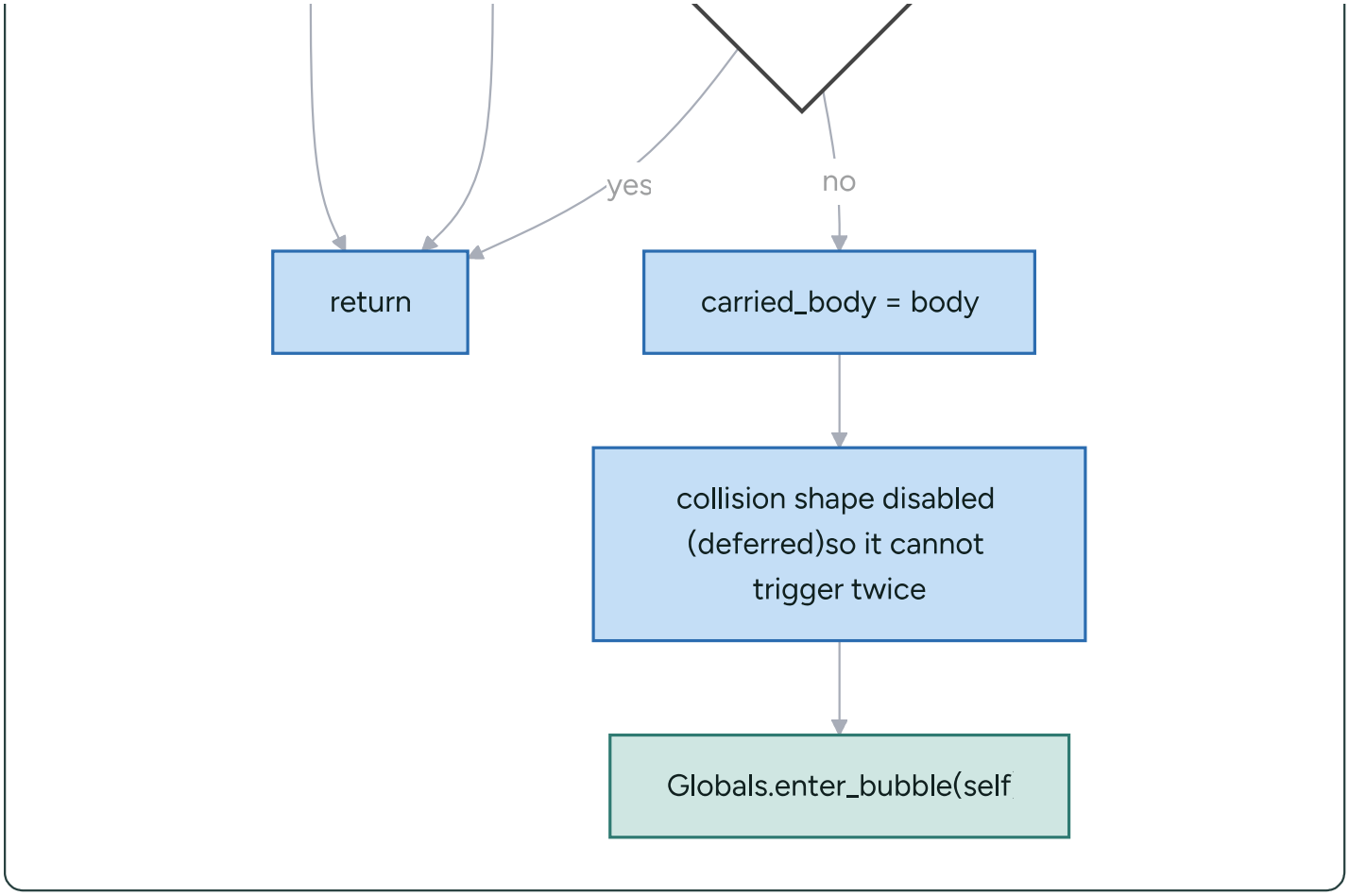
Signals	`popped` `respawned`
Exported	`respawn_time = 4.0` seconds
Variables	`spawn_position` `carried_body` `is_popped`
Nodes used	`AnimatedSprite2D` (animations `normal`, `pop`) `CollisionShape2D` `Timer` `AudioStreamPlayer`
Functions	`_ready` `_process` `_on_body_entered` `_on_body_exited` `pop` `_on_timer_timeout` `respawn` `_on_animated_sprite_2d_animation_finished`

LIFE OF ONE BUBBLE

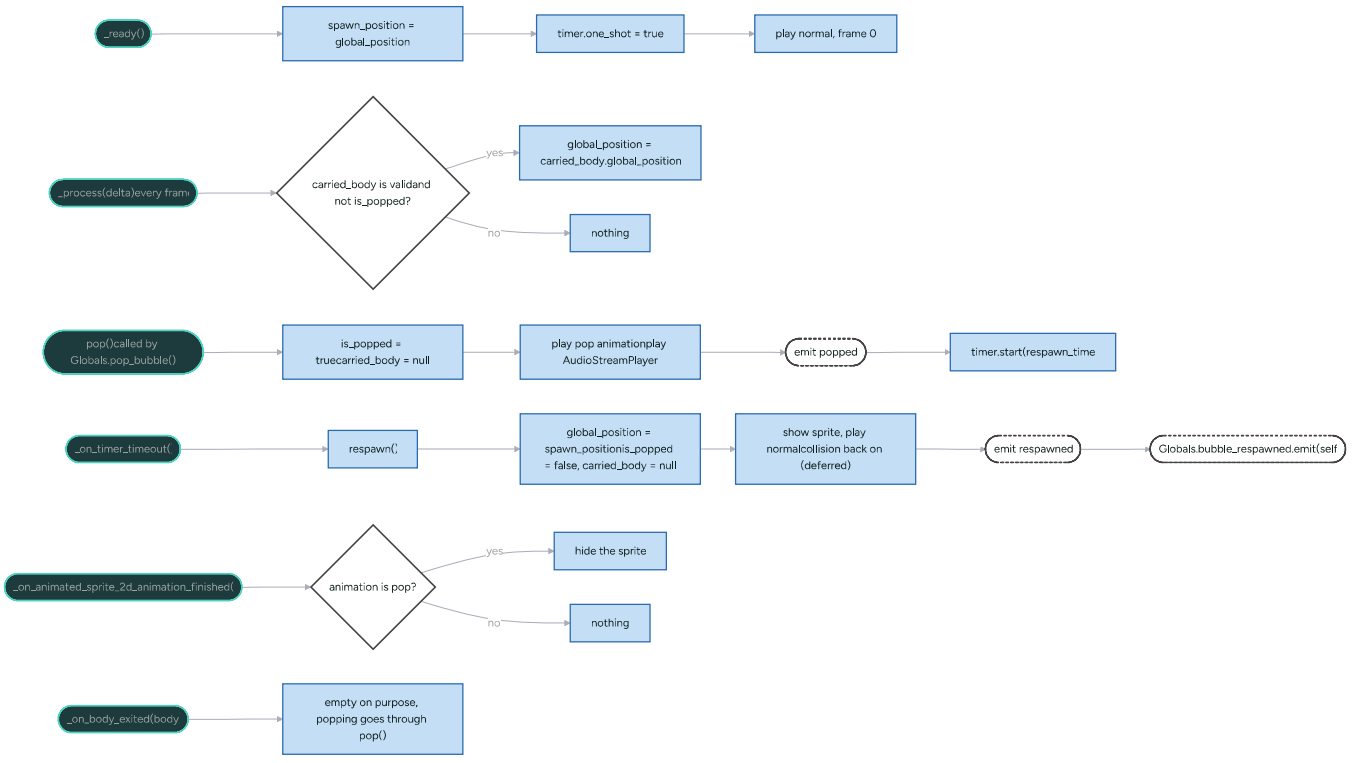


_ON_BODY_ENTERED: CAN THE FROG GET IN?





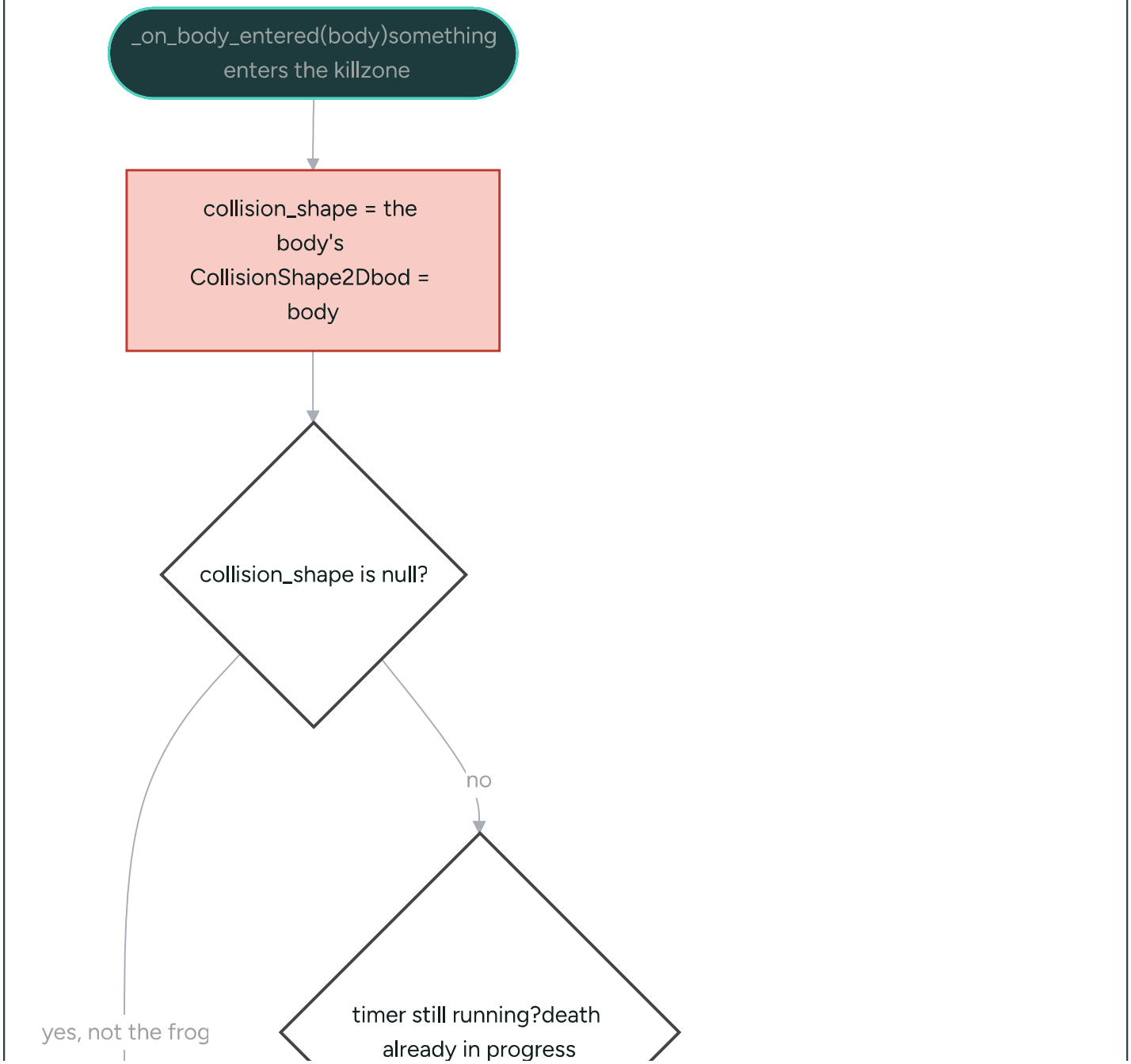
OTHER FUNCTIONS

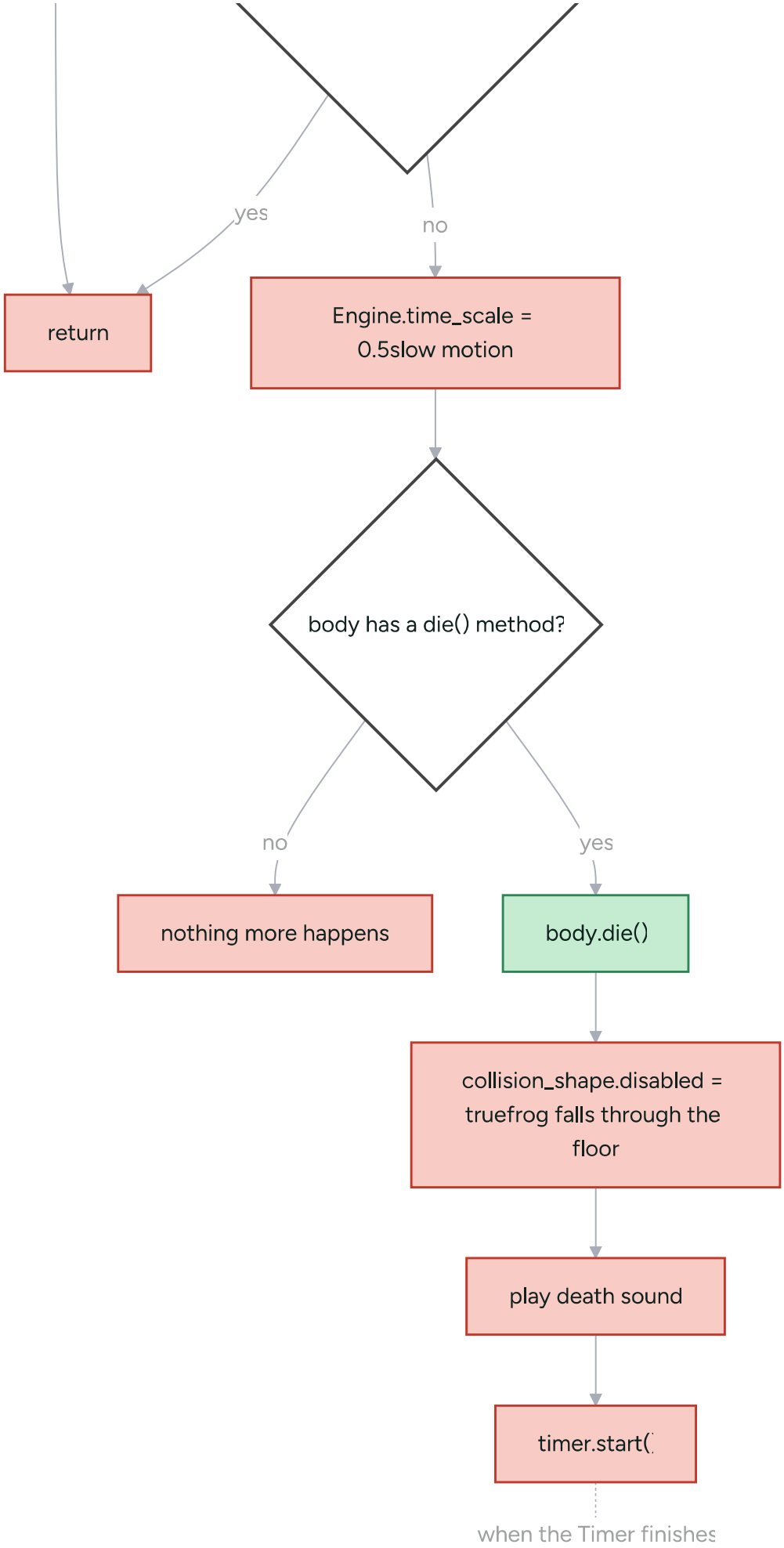


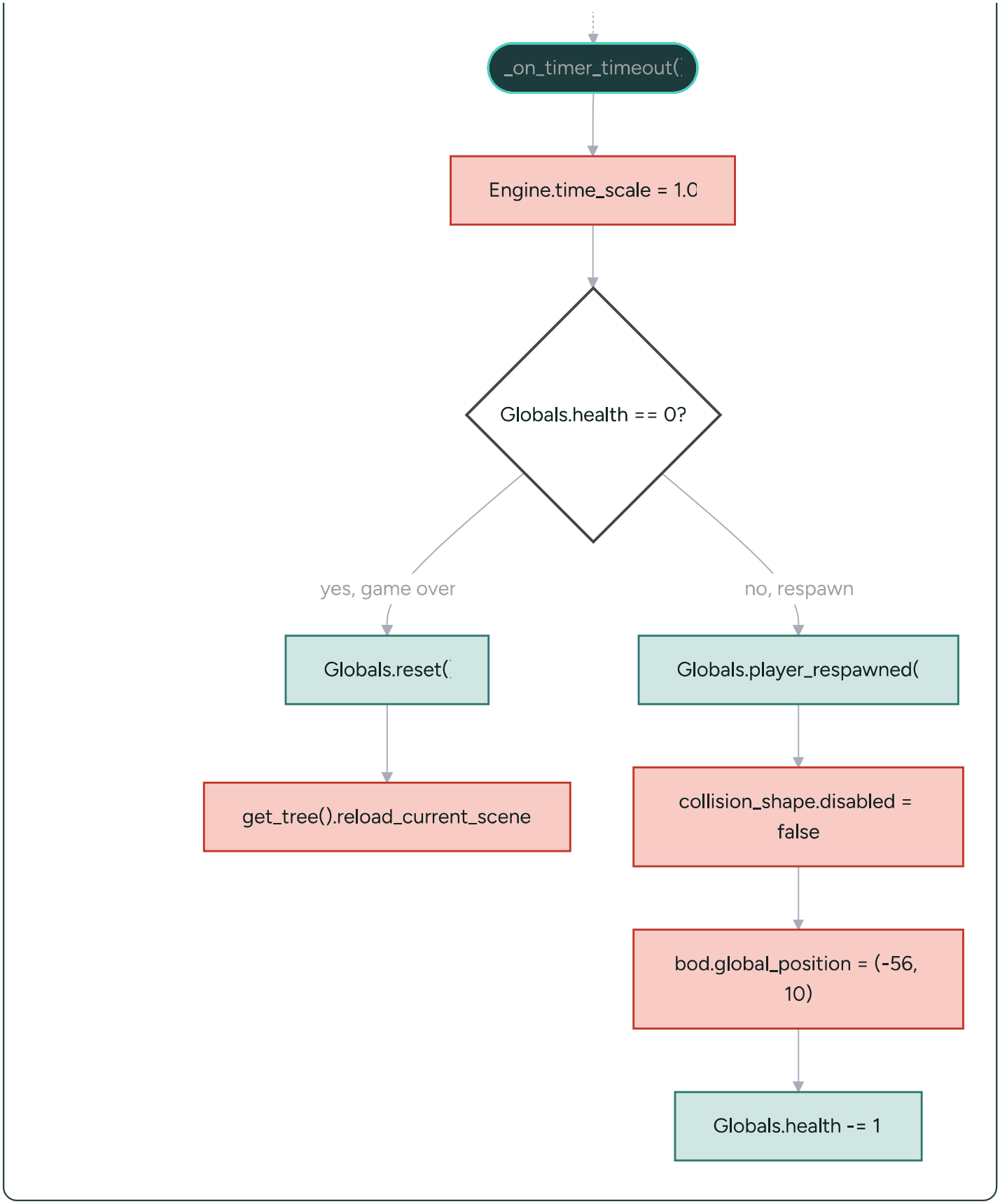
Killzone `killzone.gd`

A hazard area. When the frog enters, it triggers slow motion and the death sequence, then either respawns the frog or restarts the level.

Nodes used	`Timer` `AudioStreamPlayer2D` (`Label` and `Timer2` are declared but unused)
Variables	`collision_shape` (the frog's shape) `bod` (the frog)
Hard-coded	respawn point `Vector2(-56, 10)` and slow motion `0.5`
Functions	`_on_body_entered` `_on_timer_timeout`




yes
no
return
Engine.time_scale =
0.5slow motion
body has a die() method?
no
yes
nothing more happens
body.die()
collision_shape.disabled =
truefrog falls through the
floor
play death sound
timer.start(
when the Timer finishes

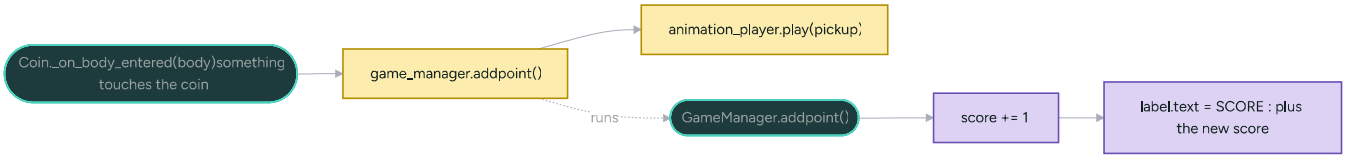


Coin and GameManager `coin.gd` `game_manager.gd`

The coin tells the game manager to add a point. The game manager owns the score and writes it

The coin tells the game manager to add a point. The game manager owns the score and writes it to the label.

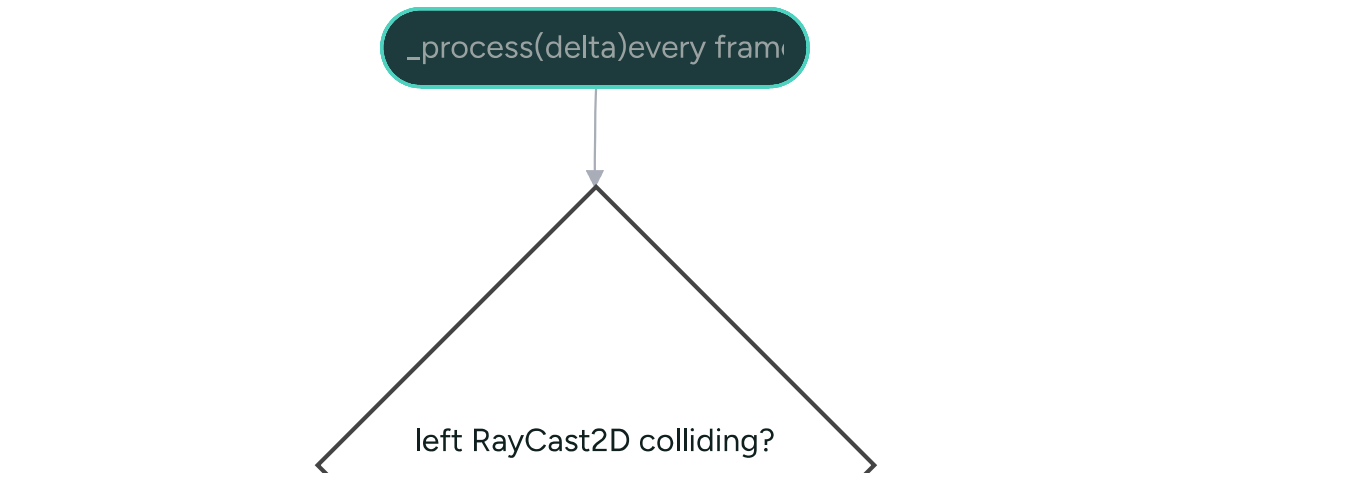
Coin extends	`Area2D`. Finds the manager through the unique name `%"Game Manager"`. Uses `AnimationPlayer` animation `pickup`. `AnimatedSprite2D` is declared but unused.
Manager extends	`Node`. Variable `score = 0`. Label found at `$"../Player1/Camera2D/Label"`. `_ready` and `_process` are empty.
Functions	`Coin._on_body_entered` `GameManager.addpoint`

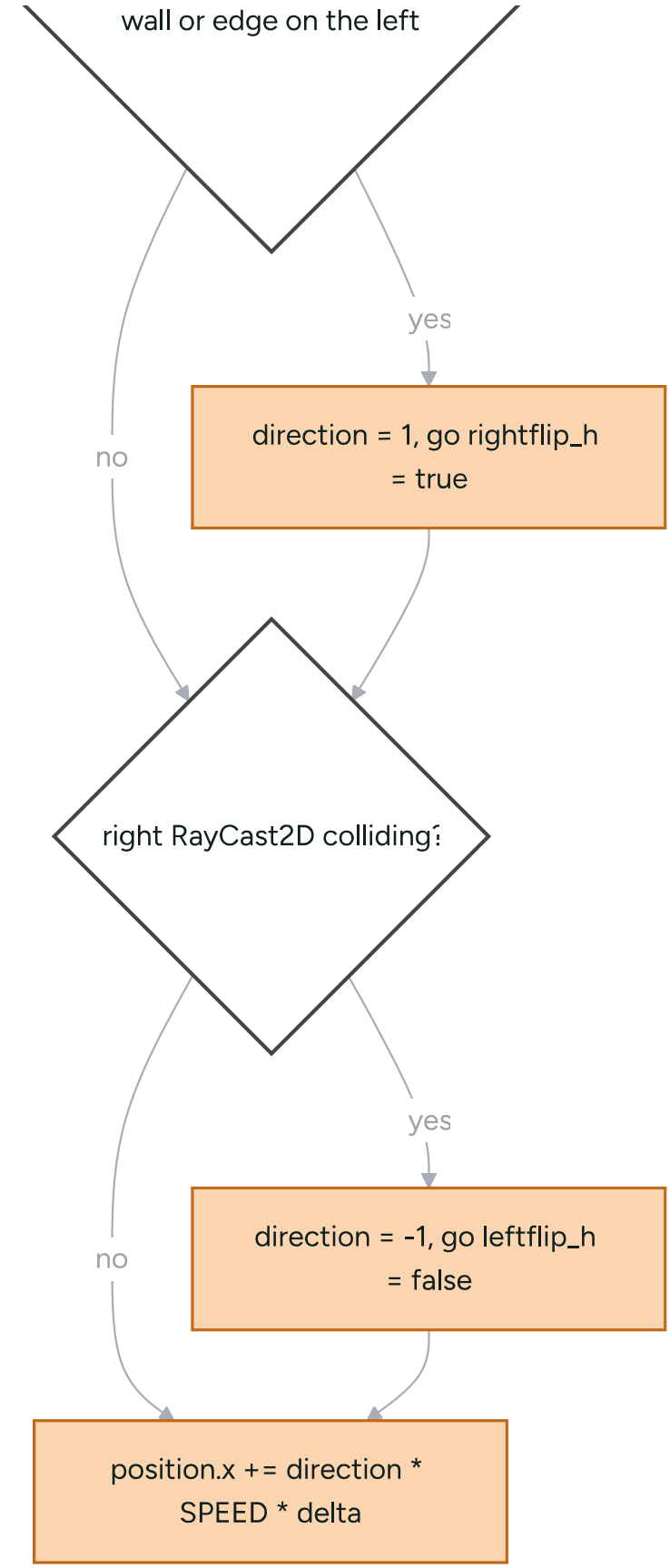


Enemy `enemy.gd`

A simple patroller. It walks sideways and turns around when a ray cast on either side hits something. It does not hurt the frog by itself.

Extends	`Node2D`
Constants and variables	`SPEED = 100` `direction = 1` (1 is right, -1 is left)
Nodes used	`RayCast2Dleft` `RayCast2Dright` `AnimatedSprite2D`
Functions	`_process` (`_ready` is empty)





Signal list

Signals you declared in code. "Nobody yet" means no script connects to it right now, so it is free

Signals you declared in code. "Nobody yet" means no script connects to it right now, so it is free to use for new features.

SIGNAL	EMITTED FROM	WHO LISTENS
`Globals.bubble_entered(bubble)`	`Globals.enter_bubble()`	`Player1._on_bu`
`Globals.bubble_popped(bubble)`	`Globals.pop_bubble()`	`Player1._on_bu`
`Globals.bubble_respawned(bubble)`	`Bubble.respawn()`	Nobody yet
`Globals.player_respawn`	`Globals.player_respawned()`, called by Killzone	`Player1._on_pl`
`Globals.bubble_state_changed(is_floating)`	`enter_bubble()` sends true, `pop_bubble()` and `reset()` send false	Nobody yet. Go floating sound o
`Bubble.popped`	`Bubble.pop()`	Nobody in code
`Bubble.respawned`	`Bubble.respawn()`	Nobody in code
`Player1.entered_bubble(bubble)`	`Player1._on_bubble_entered`	Nobody in code
`Player1.popped_bubble`	`Player1._on_bubble_popped`	`Player1._on_pc` (prints CHECK, p connected in the

Signals connected in the Godot editor

These do not appear as `connect()` calls in the code. They are inferred from the handler names, so check the Node tab, Signals, to confirm.

NODE	SIGNAL	HANDLER
Bubble (Area2D)	`body_entered`	`_on_body_entered`
Bubble (Area2D)	`body_exited`	`_on_body_exited`
Bubble Timer	`timeout`	`_on_timer_timeout`
Bubble AnimatedSprite2D	`animation_finished`	`_on_animated_sprite_2d_animation_finished`

Killzone (Area2D)	`body_entered`	`_on_body_entered`
Killzone Timer	`timeout`	`_on_timer_timeout`
Coin (Area2D)	`body_entered`	`_on_body_entered`
Player1 AnimatedSprite2D	`animation_changed`	`_on_animated_sprite_2d_animation_changed`
Player1	`popped_bubble`	`_on_popped_bubble`

Loose ends and where to build next

Score label path

`game_manager.gd` still looks for the label at `$"../Player1/Camera2D/Label"`. If you moved it into a CanvasLayer, update that path or the coin will cause a null error.

Two scores

`Globals.score` and `Globals.addpoint()` are never used. The real score lives in GameManager. Keep one of them.

Coin accepts any body

`Coin._on_body_entered` adds a point for whatever physics body touches it. Add a check that the body is the frog if other bodies can reach coins.

Hard-coded respawn point

The respawn position `(-56, 10)` is typed into the killzone. A `Marker2D` node in the level would let you move it without editing code.

Unused nodes

The killzone declares `label` and `timer_2`, and the coin declares `animated_sprite_2d`, but none are used. Remove them or give them a job.

Enemy does no damage

The enemy only patrols. To make it deadly, give it a child Area2D with `killzone.gd`.

Show lives on screen

`Globals.health` changes in the killzone only. Add a signal such as `health_changed` to Globals and update a label from it.

Bubble sounds and effects

`bubble_state_changed` and `bubble_respawned` have no listeners. They are ready for a floating sound, a trail effect or a counter.